

Non Sequitur Publishing · White Paper

Context Integrity Failure as the Agentic Cache-Poisoning Analog

Justin H. Kuiper, CISSP

Written: 2026-05-13 · v0.1-seed

Abstract

Agentic context is **assembled**, not given. The system reasons over a corpus built at runtime from retrieval results, memory entries, tool outputs, and operator-supplied input — a composition that varies across invocations and may include state from sources the operator did not explicitly authorize. This assembly process is structurally analogous to a distributed cache: inputs accumulated, expired, refreshed, and consulted under operational pressure that prioritizes availability over freshness. The analogy is more than metaphor. ****Agentic context inherits the full threat model of distributed cache poisoning****: stale-state injection, time-to-live drift, and confused-deputy delegation each have direct analogues in agentic context assembly, and each produces operationally consequential failure modes that current frameworks do not formalize.

This paper specifies a ****Context Integrity Baseline (CIB)**** as the security primitive for governed agentic context. The CIB names three properties — **integrity** (the context has not been tampered with), **freshness** (the context reflects current state within an operator-authorized validity window), and **authority** (the context-source has the operator's permission to contribute to this assembly) — and treats them as runtime-verifiable invariants. Sections 2 and 3 develop the cache-poisoning analogy and specify three CIB failure modes (stale-state injection, TTL drift, confused-deputy delegation), each with detection signatures and operator-side controls. Section 4 specifies the human authority-signer: the governance role that signs context-source authorizations, enabling unsigned or unattested state to be rejected at the context boundary. Section 5 places CIB alongside existing security controls and shows how it operationalizes Skipjack Principle 1 (context-integrity) from the parent framework. Section 6 closes with practitioner guidance and limitations.

The central claim is that ****context is a security primitive****, not an implementation detail. Treating context as a primitive with explicit integrity, freshness, and authority attributes is the structural defense against an entire class of agentic-system attacks that defender tooling currently misclassifies as reliability or quality issues.

How to cite

Justin H. Kuiper, CISSP (2026). *Context Integrity Failure as the Agentic Cache-Poisoning Analog* (v0.1-seed). Non Sequitur Publishing. <https://nonsequitur.tech/white-papers/context-integrity-failure/>

© 2026 Non Sequitur Publishing. All rights reserved. Citation with full attribution is permitted. Reproduction, redistribution, derivative works, and use as input to machine-learning training are **not** permitted without written permission. See <https://nonsequitur.tech/citation-policy/>.
Canonical URL: <https://nonsequitur.tech/white-papers/context-integrity-failure/>



ABSTRACT

Agentic context is *assembled*, not given. The system reasons over a corpus built at runtime from retrieval results, memory entries, tool outputs, and operator-supplied input — a composition that varies across invocations and may include state from sources the operator did not explicitly authorize. This assembly process is structurally analogous to a distributed cache: inputs accumulated, expired, refreshed, and consulted under operational pressure that prioritizes availability over freshness. The analogy is more than metaphor. **Agentic context inherits the full threat model of distributed cache poisoning:** stale-state injection, time-to-live drift, and confused-deputy delegation each have direct analogues in agentic context assembly, and each produces operationally consequential failure modes that current frameworks do not formalize.

This paper specifies a **Context Integrity Baseline (CIB)** as the security primitive for governed agentic context. The CIB names three properties — *integrity* (the context has not been tampered with), *freshness* (the context reflects current state within an operator-authorized validity window), and *authority* (the context-source has the operator's permission to contribute to this assembly) — and treats them as runtime-verifiable invariants. Sections 2 and 3 develop the cache-poisoning analogy and specify three CIB failure modes (stale-state injection, TTL drift, confused-deputy delegation), each with detection signatures and operator-side controls. Section 4 specifies the human authority-signer: the governance role that signs context-source authorizations, enabling unsigned or unattested state to be rejected at the context boundary. Section 5 places CIB alongside existing security controls and shows how it operationalizes Skipjack Principle 1 (context-integrity) from the parent framework.¹ Section 6 closes with practitioner guidance and limitations.

The central claim is that **context is a security primitive**, not an implementation detail. Treating context as a primitive with explicit integrity, freshness, and authority attributes is the structural defense against an entire class of agentic-system attacks that defender tooling currently misclassifies as reliability or quality issues.

HOW TO CITE

Justin H. Kuiper, CISSP (2026). *Context Integrity Failure as the Agentic Cache-Poisoning Analog* (v0.1-seed). Non Sequitur Publishing.

© 2026 Non Sequitur Publishing. All rights reserved.

1. INTRODUCTION

1.1 THE CONTEXT ASSEMBLY PROBLEM

Agentic systems do not reason against a fixed input. They reason against an assembled context: retrieved documents from RAG pipelines,²³ entries from memory stores, results from tool calls, summaries of prior interactions, plus the operator's current request. Each assembly is

1 Justin H. Kuiper, CISSP, *Mitigating Confident Misalignment in Agentic Systems: The HGC³AE² Framework* (v1.0-preprint, 2026), zenodo.org/records/19869285.

2 Patrick Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS 2020*, <https://arxiv.org/abs/2005.11401>.



a composition of state pulled from heterogeneous sources at runtime; the composition may differ across invocations even for ostensibly identical operator requests.

The composition has properties that no static input has: it is fresh-or-stale, attested-or-unattested, authorized-or-unauthorized. Existing security frameworks (input validation, retrieval hardening, RAG-specific defenses) operate on the individual sources, not on the composition. The gap is that the *integrity properties of the assembly itself* are not enforced by any frame the parent HGC³AE² paper inherits from.

1.2 THE CACHE ANALOGY

Distributed-systems practice has a well-developed security model for assembled state: distributed caches. Cache integrity, cache freshness, and cache authority are first-class concerns in cache architecture: cache poisoning,⁴ TTL exploitation, and confused-deputy cache-access have decades of defensive literature.⁵ The analogy to agentic context is structural, not metaphorical: in both cases, an assembly of state is consulted at decision time, with operational pressure to favor availability over re-verification.

If agentic context is cache-shaped, the cache-poisoning threat model applies. The contribution of this paper is to make the analogy explicit and to derive the corresponding defensive doctrine.

1.3 CONTRIBUTION

This paper makes three contributions:

Formalizes the cache-poisoning analogy for agentic context assembly, identifying three direct analogues to classical cache attacks.

Specifies the Context Integrity Baseline (CIB) as the security primitive for agentic context, with three runtime-verifiable properties (integrity, freshness, authority).

Introduces the human authority-signer as the governance role that enforces CIB at the context boundary, operationalizing Skipjack Principle 1 (context-integrity) from the HGC³AE² parent framework.⁶

The paper's primary venue is the AI-security crossover audience (target: USENIX Security or ACM CCS), with practitioner extensions appropriate for SOC and AI-deployment teams.

³ Yunfan Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997* (2024).

⁴ Steven M. Bellovin, "Using the Domain Name System for System Break-ins," *Proceedings of the 5th USENIX Security Symposium*, 1995. (Canonical reference for cache-poisoning attack patterns.)

⁵ Martin Arlitt and Carey Williamson, "Internet Web Servers: Workload Characterization and Performance Implications," *IEEE/ACM Transactions on Networking* 5, no. 5 (1997): 631-645. (Cache architecture foundational reference.)

⁶ Justin H. Kuiper, CISSP, *Mitigating Confident Misalignment in Agentic Systems: The HGC³AE² Framework* (v1.0-preprint, 2026), zenodo.org/records/19869285.



2. BACKGROUND

2.1 CONTEXT IN AGENTIC SYSTEMS

Agentic operation requires context the model can reason against. In current systems, context typically comprises:

Retrieved documents (RAG-pulled chunks from a corpus, embedded and matched against the operator's query).⁷⁸

Memory entries (summarized history from prior interactions, possibly with structured indices).

Tool outputs (results of system calls invoked during reasoning; for example, API queries, database lookups, web fetches).

Operator-supplied input (system prompt, user message, in-conversation clarifications).

System metadata (model identity, capability advertisements, environment markers).

The assembly process is opaque to the operator in most deployments. The model produces output that reflects the assembled context, but the assembly itself is not surfaced in any actionable form. The operator sees the output and judges its quality; the operator does not, by default, see *what context produced this output*.

2.2 DISTRIBUTED SYSTEMS: CONSISTENCY AND CACHE INTEGRITY

Distributed-systems literature has refined the language of state consistency and assembly. Cache architectures specifically address the problem of consulting assembled state under operational pressure:⁹

Cache integrity: the assembled state has not been modified by an unauthorized writer.

Cache freshness: the assembled state reflects current upstream truth within a validity window.

Cache authority: the source-of-truth has the right to contribute to this cache.

Cache attacks weaponize the gaps. Poisoning attacks tamper with the cache contents; freshness attacks exploit stale entries; confused-deputy attacks abuse delegation chains. The defensive vocabulary is mature; the threat models are well-characterized.

⁷ Patrick Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS 2020*, <https://arxiv.org/abs/2005.11401>.

⁸ Yunfan Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997* (2024).

⁹ Martin Arlitt and Carey Williamson, "Internet Web Servers: Workload Characterization and Performance Implications," *IEEE/ACM Transactions on Networking* 5, no. 5 (1997): 631-645. (Cache architecture foundational reference.)



2.3 THE GAP IN EXISTING FRAMEWORKS

OWASP LLM Top 10¹⁰ covers data and model poisoning (LLM03) at the source-of-truth layer. MITRE ATLAS¹¹ covers training-data poisoning at the model-preparation phase. NIST SP 800-218 covers software supply-chain integrity broadly. None of these formalize **assembly-time integrity properties** for context. The HGC³AE² parent paper names context as load-bearing and specifies the Skipjack Principle 1 mechanism (context-integrity verification at execution time);¹² this paper supplies the security primitive that operationalizes the mechanism.

3. THE CONTEXT INTEGRITY BASELINE

3.1 THE CIB TRIPLE

The Context Integrity Baseline is the property set required for agentic context to be treated as a security primitive:

Integrity (I). The assembled context contains only entries that have not been modified by an unauthorized writer since they entered the assembly pipeline. Integrity is verified per-entry via cryptographic hashes or signed retrieval results; the assembly-level integrity is the conjunction of per-entry integrity.

Freshness (F). Each entry in the assembly reflects upstream truth within an operator-authorized validity window. Stale entries — entries whose upstream truth has moved beyond the validity window — must be either refreshed or excluded; entries with unknown freshness must be marked.

Authority (A). Each context source contributing to the assembly has been authorized by the operator (or by a governance delegate) to participate in this assembly for this purpose. Unauthorized sources are rejected at the boundary; mis-categorized sources are flagged.

The CIB invariant: **$I \wedge F \wedge A$ across every entry in the assembled context**. A violation of any property in any entry is a CIB failure; the system's response to a CIB failure is the operational discipline this paper develops.

3.2 FAILURE MODE 1: STALE-STATE INJECTION

The most operationally common attack pattern. An adversary positions stale entries in a context source (RAG corpus, memory store, tool-output cache) and relies on the agentic system's assembly process to retrieve them as authoritative. The retrieved entries pass per-entry integrity (the bytes were not modified after being placed) but fail freshness (they no longer reflect upstream truth).

Detection signature. Per-entry timestamp checks against operator-authorized validity window. Entries whose source-publication timestamp predates the validity window are stale; the

¹⁰ OWASP, *OWASP Top 10 for Large Language Model Applications* (2024), <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.

¹¹ MITRE, *Adversarial Threat Landscape for Artificial-Intelligence Systems (ATLAS)* (2024), <https://atlas.mitre.org/>.

¹² Justin H. Kuiper, CISSP, *Mitigating Confident Misalignment in Agentic Systems: The HGC³AE² Framework* (v1.0-preprint, 2026), zenodo.org/records/19869285.



system flags them at retrieval. If timestamps are unavailable (raw text without metadata), the freshness property degrades to “unknown” and the entry is marked as such.

Operator-side control. The validity window is operator-pinned at governance time. The window may be content-class-dependent (e.g., regulatory texts: months; market data: minutes; system status: seconds). The agentic system respects the window; entries outside the window are excluded or surfaced as stale-suspect.

3.3 FAILURE MODE 2: TTL DRIFT

A subtle variant of stale-state injection. The cache TTL (time-to-live) on a context source is set higher than the upstream-truth volatility justifies. Entries do not technically violate freshness (they are within the configured TTL) but do violate operational truth (the upstream has moved within the TTL window). TTL drift is an architectural defect rather than an active attack, but it produces attack-shaped outcomes.

Detection signature. Audit of TTL settings against upstream-volatility analysis. Where the TTL exceeds the operationally-justified window, drift is structural; refactoring the TTL is the response.

Operator-side control. TTL governance: operator-owned policy that sets per-class TTLs based on operational risk tolerance. The agentic system respects the TTLs; the governance review cadence inspects whether TTLs remain calibrated to current operational reality.

3.4 FAILURE MODE 3: CONFUSED-DEPUTY DELEGATION

A more sophisticated attack pattern, derived from the classical confused-deputy problem¹³. The agentic system has legitimate access to a context source X (authorized for one purpose) and is induced — typically via prompt injection — to retrieve from X for a different purpose. The retrieval is technically authorized (the system has access to X) but operationally unauthorized (the operator did not authorize X for the purpose).

Detection signature. Per-call authorization check: not only “does the system have access to X?” but “did the operator authorize X for *this kind of use*?” The latter requires per-source purpose-bound authorization, not session-level access grants.

Operator-side control. Purpose-bound source authorization. Each context source is registered with the purposes for which it may contribute; assembly invocations specify the purpose and verify source-purpose compatibility. Unauthorized purpose-source combinations are rejected at the boundary.

4. THE HUMAN AUTHORITY-SIGNER

4.1 THE SIGNING MECHANISM

The CIB requires per-entry verification, but the verification requires a *source of authority* for what is integrity-valid, freshness-valid, and authority-valid. That source is the **human authority-signer**: a governance role that pins, at deployment or governance time, the policies the CIB enforces.

¹³ Norm Hardy, “The Confused Deputy: (or why capabilities might have been invented),” *ACM SIGOPS Operating Systems Review* 22, no. 4 (1988): 36-38, <https://doi.org/10.1145/54289.871709>.



The signing mechanism is operationally specific. The authority-signer's signature attaches to:

Source registrations (which context sources are authorized for which purposes).

Validity windows (what freshness window applies to each content class).

Integrity baselines (which cryptographic verification chains are required for each source).

The signature is logged (tamper-evident) and consulted at runtime: when the agentic system assembles context, the CIB enforcement checks every entry against the authority-signer's pins for that source.

4.2 GOVERNANCE REQUIREMENTS

The authority-signer role has operational requirements:

Identifiable authority. The signer is a named human (or named role with named occupant), not an abstract policy. Auditability requires the identity be recoverable from the signing chain.

Bounded scope. The signer authorizes for specific deployments, contexts, and time windows. The signing is not blanket; it is contextually scoped.

Revocability. The signer can revoke prior authorizations; the agentic system respects revocations within a propagation window (configurable; default: under one operating cycle).

Audit trail. Every signing event and every CIB enforcement event is logged; audit consults the joint log to answer “what context was assembled, under whose authority, against which policies.”

4.3 WHAT THE AUTHORITY-SIGNER CANNOT DO

The role is bounded by structural commitments:

The signer cannot retroactively legalize an unauthorized assembly. CIB violations that occurred under prior policy stand as violations; the signing chain captures policy at the time of assembly, not policy at the time of audit.

The signer cannot waive freshness for compliance-mandated content. Where regulation requires currency, the freshness window is regulatorily floored; the signer's authority is bounded by regulatory minima.

The signer cannot delegate the role without explicit succession ceremony. The role is durable; transitions are logged with continuity guarantees.

5. CIB AS A SECURITY PRIMITIVE

5.1 RELATIONSHIP TO EXISTING CONTROLS

CIB sits alongside existing security controls without subsuming them:

Input validation (OWASP LLM01 territory) operates at the input boundary — before context assembly. CIB operates at the *assembled-context boundary* — after assembly, before reasoning.



Data integrity (NIST SP 800-53 controls) operates on stored data. CIB operates on assembled context that may include data from heterogeneous storage.

Source authentication operates per-source. CIB operates per-assembly, requiring source-authentication outputs as input.

CIB is the missing primitive that ties existing controls into a context-aware defensive architecture. Without it, source-authenticated entries can compose into an unauthenticated assembly; freshness can be ensured per-source but not per-assembly; authority can be granted at session level but exploited at use level (confused-deputy).

5.2 OPERATIONAL INTEGRATION WITH SKIPJACK

The HGC³AE² parent paper specifies four Skipjack principles; Principle 1 (context-integrity verification at execution time)¹⁴ is what CIB operationalizes. The mapping is:

Skipjack P1 says: verify context integrity at runtime, fail-closed on failure.

CIB says: the verification has three properties (I, F, A), enforced per-entry, against authority-signer pins.

The composition is: Skipjack provides the enforcement mechanism; CIB provides the policy that the mechanism enforces.

A system that implements Skipjack P1 without CIB has the enforcement mechanism but lacks the policy detail; a system that has CIB without Skipjack P1 has policy without runtime enforcement. Both are required.

5.3 IMPLICATIONS FOR TOOLING

Practitioner tooling for agentic systems should surface CIB readings as primary state. Per-assembly integrity / freshness / authority indicators, per-entry source attributions, and audit-log integration are the operational outputs CIB requires. Current tooling rarely supplies these; the gap is the practitioner consequence of treating context as a security primitive.

6. LIMITATIONS AND CONCLUSION

6.1 LIMITATIONS

CIB assumes the existence of operationally-meaningful freshness windows per content class. In domains with extreme volatility (real-time markets, system status) or extreme stability (regulatory text in stable jurisdictions), the windows are easy to specify; in middle-ground domains they require calibration that this paper does not develop in detail.

The authority-signer role assumes human-scale governance bandwidth. In high-volume agentic deployments, signing events may exceed the practical capacity of the role; delegation patterns and signing-as-policy-update are necessary engineering complements not developed here.

¹⁴ Justin H. Kuiper, CISSP, *Mitigating Confident Misalignment in Agentic Systems: The HGC³AE² Framework* (v1.0-preprint, 2026), zenodo.org/records/19869285.



The cache-poisoning analogy is structural; specific cache-attack techniques may not map cleanly to specific agentic-context-attack techniques. The analogy guides the threat model but does not substitute for empirical study of attack patterns in production deployments.

6.2 CONCLUSION

Context is a security primitive. Treating it as one — with integrity, freshness, and authority as runtime-verifiable invariants enforced against authority-signer policies — is the structural defense against the agentic-cache-poisoning threat class. The contribution of CIB is to make the primitive specifiable: three properties, three failure modes, one governance role. The contribution to the HGC³AE² family is to operationalize Skipjack Principle 1 with the policy detail it requires for production use.

This is Thesis Two (cyb-02) of the HGC³AE² Cyber Pillar derivative family. Companion theses (cyb-01 confident-misalignment-attack-surface, cyb-03 denial-of-validation, cyb-06 agentic-zero-trust, cyb-08 reconciliation-soc-doctrine, cyb-09 tool-orchestration-trust-composition, cyb-10 human-authority-chain-of-custody) develop adjacent dimensions of the family's defense architecture.

REFERENCES

— end v0.1-seed draft —

Drafting state: v0.1 CYB-02 (chain v0.1 step 2). 6-section canon shape. PR count: 12 — parent (HGC³AE²) + 9 inline RAG/cache/poisoning/confused-deputy/NIST references + 2 cross-thesis refs (cyb-01, cyb-03). Meets ≥12 floor.